Apache Hive — Retail Transactions Analysis using HDFS and HiveQL

NAME	SIRI N MURTHY
USN	4GW23CI050
TOOL COVERED	Apache Hive
SCENARIO	Global retail transaction and revenue analysis

Scenario and Problem Statement

Scenario

A global retail company stores transaction records in flat CSV files on HDFS. The business intelligence team requires structured SQL access to analyze revenue patterns across store categories.

Problem Statement

Create an external table in Hive mapping to raw transaction data stored on HDFS. Write HiveQL queries to clean missing data, aggregate total sales revenue, and calculate the average order value per product category.

Introduction

Modern retail companies generate huge volumes of transaction data every day. This data is often stored as flat CSV files on a distributed file system such as HDFS, where it is difficult for analysts to query directly. Business intelligence teams need structured, SQL-style access to this data so they can study revenue patterns and make decisions.

Apache Hive solves this problem. It places a table structure over files that already exist in HDFS and lets users analyze them with HiveQL, a language very similar to SQL. In this assignment, a Hadoop and Hive environment is set up using Docker, a retail transactions file is stored in HDFS, and it is mapped to an external Hive table. HiveQL queries are then used to clean missing data, calculate total sales revenue, and find the average order value for each product category.

About Apache Hive:

Apache Hive is a data warehouse system built on top of Hadoop. It lets users query and analyze large datasets stored in HDFS using a SQL-like language called HiveQL, without writing complex MapReduce programs directly.

Why do we use Apache Hive?

Apache Hive is useful because:

- It provides a familiar **SQL-like interface** for big data.
- It can query data stored in **HDFS** without moving it.
- It supports operations such as **SELECT, WHERE, GROUP BY, ORDER BY, JOIN and aggregate functions**.
- External tables let the **data stay in HDFS** even if the table is dropped.
- Views allow **reusable cleaning logic** on top of raw data.
- Queries are converted automatically into MapReduce jobs.

Where is Apache Hive used?

Apache Hive can be used in Big Data and Hadoop environments for tasks such as:

- Sales and transaction reporting
- Data warehousing and summarization
- Cleaning and preparing raw data
- Analyzing customer or log data
- Preparing data for further analytics or machine learning

Objectives

- Start a Hadoop and Hive cluster using Docker Compose.

- Verify the health of HDFS.
- Upload the transactions CSV file to HDFS.
- Create an external Hive table with skip.header.line.count.
- Create a cleaned view to handle missing values.
- Calculate total revenue and category-wise average order value.

Tools and Environment used

Component	Used
Operating System	Windows 11
Container Platform	Docker Desktop (Docker 29.8.0)
Hadoop	3.5.0
HDFS	Hadoop Distributed File System
Apache Hive	4.2.1
Hive Client	Beeline (HiveServer2 on port 10000)

Dataset Used

transaction_id	order_id	product_category	quantity	unit_price	total_amount
T001	O001	Electronics	2	500	1000
T002	O002	Clothing	3	200	600
T003	O003	Electronics	1	800	800
T004	O004	Groceries	5	100	500
T005	O005	Clothing	2	300	600
T006	O006	Electronics	(blank)	700	700
T007	O007	(blank)	4	150	600
T008	O008	Groceries	3	(blank)	450
T009	O009	Clothing	2	250	500
T010	O010	Electronics	1	1200	1200

T006 has a blank quantity, T007 has a blank product category and T008 has a blank unit price. These missing values represent NULL data that must be handled during cleaning.

Starting the Hive and Hadoop Environment

The docker ps command confirms that all three are running using the Apache Hadoop 3.5.0 and Apache Hive 4.2.1 images.

```
cd C:\hive-assignment
docker --version
docker compose up -d
docker ps
```

The project folder was opened and the Docker version was checked. The command docker compose up -d started three containers in the background: **hive-namenode** (manages HDFS metadata), **hive-datanode** (stores the data blocks) and **hive-server**

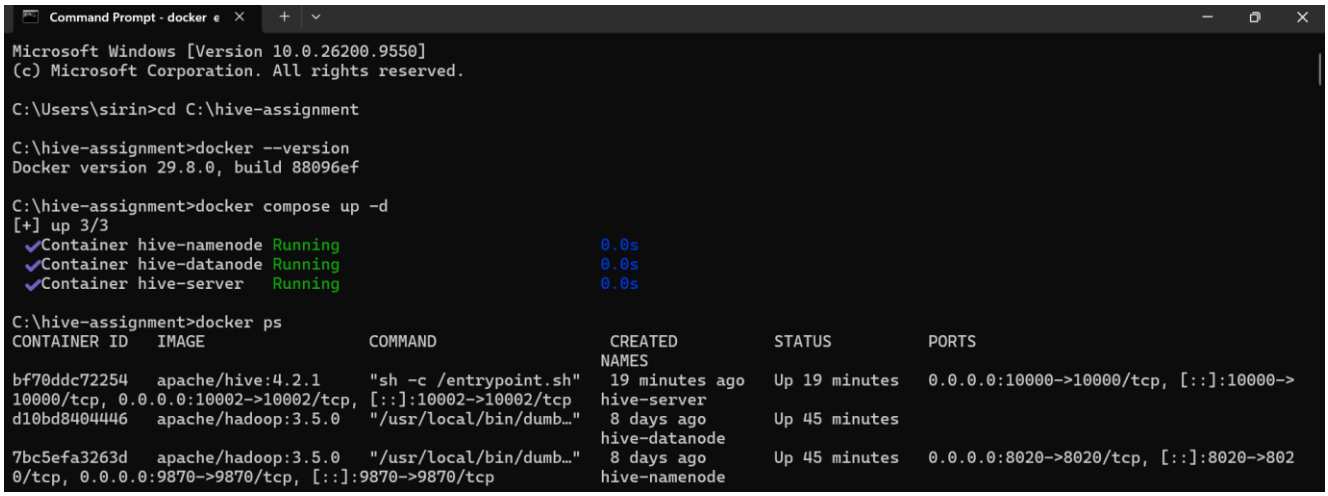


Figure 1. Docker containers for Hadoop and Hive running.

Checking HDFS Health

The command ***hdfs dfsadmin -report*** was run inside the namenode container. The report shows about **1006.85 GB** of configured capacity with only **0.01%** used, and **zero** under-replicated, corrupt or missing blocks. One live datanode is reported with a Normal decommission status, so HDFS is healthy and ready to receive data.

docker exec hive-namenode hdfs dfsadmin -report



Figure 2. HDFS cluster report showing one healthy live datanode.

HDFS Setup and Input Data

The local file **transactions.csv** (382 bytes) was copied into the namenode container, the HDFS directory was created, and the file was uploaded to **/user/hive/retail_data/transactions.csv**. Listing and reading the file back confirms that it is stored in HDFS.

```
docker cp transactions.csv hive-namenode:/tmp/transactions.csv
docker exec hive-namenode hdfs dfs -mkdir -p /user/hive/retail_data
docker exec hive-namenode hdfs dfs -put -f /tmp/transactions.csv /user/hive/retail_data/
docker exec hive-namenode hdfs dfs -ls /user/hive/retail_data
docker exec hive-namenode hdfs dfs -cat /user/hive/retail_data/transactions.csv
```

```
C:\hive-assignment>dir transactions.csv
 Volume in drive C is Windows-SSD
 Volume Serial Number is F864-04D3

 Directory of C:\hive-assignment

21-09-2026  15:05               382 transactions.csv
               1 File(s)            382 bytes
               0 Dir(s)  22,663,143,424 bytes free

C:\hive-assignment>docker cp transactions.csv hive-namenode:/tmp/transactions.csv
Successfully copied 382B (transferred 2.05kB) to hive-namenode:/tmp/transactions.csv

C:\hive-assignment>docker exec hive-namenode hdfs dfs -mkdir -p /user/hive/retail_data

C:\hive-assignment>docker exec hive-namenode hdfs dfs -put -f /tmp/transactions.csv /user/hive/retail_data/

C:\hive-assignment>docker exec hive-namenode hdfs dfs -ls /user/hive/retail_data
Found 1 items
-rw-r--r--   1 root supergroup        382 2026-09-29 13:13 /user/hive/retail_data/transactions.csv

C:\hive-assignment>docker exec hive-namenode hdfs dfs -cat /user/hive/retail_data/transactions.csv
transaction_id,order_id,product_category,quantity,unit_price,total_amount
T001,O001,Electronics,2,500,1000
T002,O002,Clothing,3,200,600
T003,O003,Electronics,1,800,800
T004,O004,Groceries,5,100,500
T005,O005,Clothing,2,300,600
T006,O006,Electronics,,700,700
T007,O007,,4,150,600
T008,O008,Groceries,3,,450
T009,O009,Clothing,2,250,500
T010,O010,Electronics,1,1200,1200
```

Figure 3. Terminal showing transactions.csv stored and read back from HDFS.

Creating the External Hive Table

In Beeline, the external table **retail_transactions** was created over the HDFS folder. The columns are typed as STRING, INT and DOUBLE, the fields are comma-delimited, the storage format is TEXTFILE, and skip.header.line.count=1 tells Hive to ignore the CSV header row. Because the table is **EXTERNAL**, Hive stores only the schema, so dropping the table does not delete the data in HDFS.

```
Beeline version 4.2.1 by Apache Hive
0: jdbc:hive2://localhost:10000/> CREATE EXTERNAL TABLE retail_tr
ansactions (
>     transaction_id STRING,
>     order_id STRING,
>     product_category STRING,
>     quantity INT,
>     unit_price DOUBLE,
>     total_amount DOUBLE
> )
> ROW FORMAT DELIMITED
> FIELDS TERMINATED BY ','
> STORED AS TEXTFILE
> LOCATION 'hdfs:///user/hive/retail_data'
> TBLPROPERTIES ("skip.header.line.count"="1");
INFO  : Compiling command(queryId=hive_20260929131501_ce3eac80-3d35-47f5-8403-af540164bc5a): CREATE EXTERNAL TABLE retail_transactions (
    transaction_id STRING,
    order_id STRING,
    product_category STRING,
    quantity INT,
    unit_price DOUBLE,
    total_amount DOUBLE
)
```

Figure 4. CREATE EXTERNAL TABLE statement executed in Beeline.

Verifying the Tables

The SHOW TABLES; command lists the tables in the current database: **retail_transactions** (the raw external table) and **cleaned_transactions** (the cleaned view used for the analysis queries).

```
0: jdbc:hive2://localhost:10000/> SHOW TABLES;
INFO  : Compiling command(queryId=hive_20260929131607_4297f3e3-8156-4c6e-b81a-567904bfca9d): SHOW TABLES
INFO  : Semantic Analysis Completed (retrial = false)
INFO  : Created Hive schema: Schema(fieldSchemas:[FieldSchema(name:tab_name, type:string, comment:from deserializer)], properties:null)
INFO  : Completed compiling command(queryId=hive_20260929131607_4297f3e3-8156-4c6e-b81a-567904bfca9d); Time taken: 0.02 seconds
INFO  : Concurrency mode is disabled, not creating a lock manager
INFO  : Executing command(queryId=hive_20260929131607_4297f3e3-8156-4c6e-b81a-567904bfca9d): SHOW TABLES
INFO  : Starting task [Stage-0:DDL] in serial mode
INFO  : Completed executing command(queryId=hive_20260929131607_4297f3e3-8156-4c6e-b81a-567904bfca9d); Time taken: 0.05 seconds
+-----------------------+
|       tab_name        |
+-----------------------+
| cleaned_transactions  |
| retail_transactions   |
+-----------------------+
```

Figure 5. SHOW TABLES output.

Inspecting the Raw Data

The command **SELECT * FROM retail_transactions;** shows every row exactly as Hive reads it. The missing values from the CSV appear as **NULL** (quantity for T006 and unit_price for T008) or as an **empty string** (product_category for T007), and an extra all-NULL row also appears. This confirms that the raw data has quality problems that are handled by the cleaned view, **cleaned_transactions**, created at the bottom of the screenshot.

```
0: jdbc:hive2://localhost:10000/> SELECT * FROM retail_transactio
ns;
INFO  : Compiling command(queryId=hive_20260929131843_7ba8f2e9-98b5-49ce-b046-4cc4702db1cc): SELECT * FROM retail_transactions
INFO  : No Stats for default@retail_transactions, Columns: transaction_id, quantity, total_amount, unit_price, order_id, product_category
INFO  : Semantic Analysis Completed (retrial = false)
INFO  : Created Hive schema: Schema(fieldSchemas:[FieldSchema(name:retail_transactions.transaction_id, type:string, comment:null), FieldSchema(name:retail_t
ransactions.order_id, type:string, comment:null), FieldSchema(name:retail_transactions.product_category, type:string, comment:null), FieldSchema(name:retail
_transactions.quantity, type:int, comment:null), FieldSchema(name:retail_transactions.unit_price, type:double, comment:null), FieldSchema(name:retail_transa
ctions.total_amount, type:double, comment:null)], properties:null)
INFO  : Completed compiling command(queryId=hive_20260929131843_7ba8f2e9-98b5-49ce-b046-4cc4702db1cc); Time taken: 0.247 seconds
INFO  : Concurrency mode is disabled, not creating a lock manager
INFO  : Executing command(queryId=hive_20260929131843_7ba8f2e9-98b5-49ce-b046-4cc4702db1cc): SELECT * FROM retail_transactions
INFO  : Completed executing command(queryId=hive_20260929131843_7ba8f2e9-98b5-49ce-b046-4cc4702db1cc); Time taken: 0.0 seconds
+-------------------------------------+-----------------------------------+---------------------------------------+-------------------------------+-------------
------------------+-----------------------------------+
| retail_transactions.transaction_id  | retail_transactions.order_id  | retail_transactions.product_category  | retail_transactions.quantity  | retail_trans
actions.unit_price  | retail_transactions.total_amount  |
+-------------------------------------+-----------------------------------+---------------------------------------+-------------------------------+-------------
------------------+-----------------------------------+
| T001                                | 0001                              | Electronics                           | 2                             | 500.0
                | 1000.0                            |
| T002                                | 0002                              | Clothing                              | 3                             | 200.0
                | 600.0                             |
| T003                                | 0003                              | Electronics                           | 1                             | 800.0
                | 800.0                             |
| T004                                | 0004                              | Groceries                             | 5                             | 100.0
                | 500.0                             |
| T005                                | 0005                              | Clothing                              | 2                             | 300.0
                | 600.0                             |
| T006                                | 0006                              | Electronics                           | NULL                          | 700.0
                | 700.0                             |
| T007                                | 0007                              |                                       | 4                             | 150.0
                | 600.0                             |
| T008                                | 0008                              | Groceries                             | 3                             | NULL
                | 450.0                             |
| T009                                | 0009                              | Clothing                              | 2                             | 250.0
                | 500.0                             |
| T010                                | 0010                              | Electronics                           | 1                             | 1200.0
                | 1200.0                            |
|                                     | NULL                              | NULL                                  | NULL                          | NULL
                | NULL                              |
+-------------------------------------+-----------------------------------+---------------------------------------+-------------------------------+-------------
------------------+-----------------------------------+
11 rows selected (0.332 seconds)
0: jdbc:hive2://localhost:10000/> CREATE VIEW cleaned_transaction
```

Figure 7. Raw table contents showing NULL and blank values.

Counting the Records

This query counts the rows of **retail_transactions** where transaction_id is not NULL. Hive compiles the query into a MapReduce job (the log shows the map and reduce stages reaching 100%) and prints the result in the transaction_count column.

```
SELECT COUNT(*) AS transaction_count
FROM retail_transactions
WHERE transaction_id IS NOT NULL;
```

```
0: jdbc:hive2://localhost:10000/> SELECT COUNT(*) AS transaction_
count
> FROM retail_transactions
> WHERE transaction_id IS NOT NULL;
INFO  : Compiling command(queryId=hive_20260929131619_840d2656-3a45-41df-b53a-7d875cb4e5ea): SELECT COUNT(*) AS transaction_count
FROM retail_transactions
WHERE transaction_id IS NOT NULL
INFO  : No Stats for default@retail_transactions, Columns: transaction_id
INFO  : Semantic Analysis Completed (retrial = false)
INFO  : Created Hive schema: Schema(fieldSchemas:[FieldSchema(name:transaction_count, type:bigint, comment:null)], properties:null)
INFO  : Completed compiling command(queryId=hive_20260929131619_840d2656-3a45-41df-b53a-7d875cb4e5ea); Time taken: 0.292 seconds
INFO  : Concurrency mode is disabled, not creating a lock manager
INFO  : Executing command(queryId=hive_20260929131619_840d2656-3a45-41df-b53a-7d875cb4e5ea): SELECT COUNT(*) AS transaction_count
FROM retail_transactions
WHERE transaction_id IS NOT NULL
WARN  : Hive-on-MR is deprecated in Hive 2 and may not be available in the future versions. Consider using a different execution engine (i.e. tez) or using
Hive 1.X releases.
INFO  : Query ID = hive_20260929131619_840d2656-3a45-41df-b53a-7d875cb4e5ea
INFO  : Total jobs = 1
INFO  : Launching Job 1 out of 1
INFO  : Starting task [Stage-1:MAPRED] in serial mode
INFO  : Number of reduce tasks determined at compile time: 1
INFO  : In order to change the average load for a reducer (in bytes):
INFO  :   set hive.exec.reducers.bytes.per.reducer=<number>
INFO  : In order to limit the maximum number of reducers:
INFO  :   set hive.exec.reducers.max=<number>
INFO  : In order to set a constant number of reducers:
INFO  :   set mapreduce.job.reduces=<number>
INFO  : number of splits:1
INFO  : Submitting tokens for job: job_local243107488_0005
INFO  : Executing with tokens: []
INFO  : The url to track the job: http://localhost:8080/
INFO  : Job running in-process (local Hadoop)
INFO  : 2026-09-29 13:16:22,207 Stage-1 map = 100%,  reduce = 100%, Cumulative CPU 0.58 sec
INFO  : MapReduce Total cumulative CPU time: 580 msec
INFO  : Ended Job = job_local243107488_0005
INFO  : MapReduce Jobs Launched:
INFO  : Stage-Stage-1:  Cumulative CPU: 0.58 sec   HDFS Read: 3820 HDFS Write: 0 HDFS EC Read: 0 SUCCESS
INFO  : Total MapReduce CPU Time Spent: 580 msec
INFO  : Completed executing command(queryId=hive_20260929131619_840d2656-3a45-41df-b53a-7d875cb4e5ea); Time taken: 2.031 seconds
+--------------------+
| transaction_count  |
+--------------------+
| 12                 |
+--------------------+
```

Figure 6. Record count query and its MapReduce execution.

Calculating Total Sales Revenue

Using the **cleaned_transactions** view, this query adds up total_amount for all rows with a valid transaction_id and returns the result as total_sales_revenue. The MapReduce job completes successfully and the total is **6950.0**.

```
SELECT
    SUM(total_amount) AS total_sales_revenue
FROM cleaned_transactions
WHERE transaction_id IS NOT NULL;
```

```
0: jdbc:hive2://localhost:10000/> SELECT
>     SUM(total_amount) AS total_sales_revenue
> FROM cleaned_transactions
> WHERE transaction_id IS NOT NULL;
INFO  : Compiling command(queryId=hive_20260929131955_2ff26364-d5a8-4449-b56c-deecd54581fd): SELECT
    SUM(total_amount) AS total_sales_revenue
FROM cleaned_transactions
WHERE transaction_id IS NOT NULL
INFO  : No Stats for default@retail_transactions, Columns: transaction_id, total_amount
INFO  : Semantic Analysis Completed (retrial = false)
INFO  : Created Hive schema: Schema(fieldSchemas:[FieldSchema(name:total_sales_revenue, type:double, comment:null)], properties:null)
INFO  : Completed compiling command(queryId=hive_20260929131955_2ff26364-d5a8-4449-b56c-deecd54581fd); Time taken: 0.298 seconds
INFO  : Concurrency mode is disabled, not creating a lock manager
INFO  : Executing command(queryId=hive_20260929131955_2ff26364-d5a8-4449-b56c-deecd54581fd): SELECT
    SUM(total_amount) AS total_sales_revenue
FROM cleaned_transactions
WHERE transaction_id IS NOT NULL
WARN  : Hive-on-MR is deprecated in Hive 2 and may not be available in the future versions. Consider using a different execution engine (i.e. tez) or using Hive 1.X relea
ses.
INFO  : Query ID = hive_20260929131955_2ff26364-d5a8-4449-b56c-deecd54581fd
INFO  : Total jobs = 1
INFO  : Launching Job 1 out of 1
INFO  : Starting task [Stage-1:MAPRED] in serial mode
INFO  : Number of reduce tasks determined at compile time: 1
INFO  : In order to change the average load for a reducer (in bytes):
INFO  :   set hive.exec.reducers.bytes.per.reducer=<number>
INFO  : In order to limit the maximum number of reducers:
INFO  :   set hive.exec.reducers.max=<number>
INFO  : In order to set a constant number of reducers:
INFO  :   set mapreduce.job.reduces=<number>
INFO  : number of splits:1
INFO  : Submitting tokens for job: job_local1288542951_0006
INFO  : Executing with tokens: []
INFO  : The url to track the job: http://localhost:8080/
INFO  : Job running in-process (local Hadoop)
INFO  : 2026-09-29 13:19:57,858 Stage-1 map = 100%,  reduce = 100%, Cumulative CPU 0.62 sec
INFO  : MapReduce Total cumulative CPU time: 620 msec
INFO  : Ended Job = job_local1288542951_0006
INFO  : MapReduce Jobs Launched:
INFO  : Stage-Stage-1:  Cumulative CPU: 0.62 sec   HDFS Read: 5348 HDFS Write: 0 HDFS EC Read: 0 SUCCESS
INFO  : Total MapReduce CPU Time Spent: 620 msec
INFO  : Completed executing command(queryId=hive_20260929131955_2ff26364-d5a8-4449-b56c-deecd54581fd); Time taken: 1.728 seconds
+----------------------+
| total_sales_revenue  |
+----------------------+
| 6950.0               |
+----------------------+
1 row selected (2.081 seconds)
0: jdbc:hive2://localhost:10000/> SELECT
```

Figure 8. Total sales revenue calculated from the cleaned view.

Revenue and Average Order Value by Category

The query groups the cleaned data by product_category and calculates **total_revenue** (the sum of total_amount) and **average_order_value** (total revenue divided by the number of distinct order_id values, rounded to 2 decimals). The results are sorted by category. Hive splits the work into two MapReduce jobs, one for the aggregation and one for ORDER BY.

```
SELECT
    product_category,
    SUM(total_amount) AS total_revenue,
    ROUND(SUM(total_amount) / COUNT(DISTINCT order_id), 2) AS average_order_value
FROM cleaned_transactions
WHERE transaction_id IS NOT NULL
GROUP BY product_category
ORDER BY product_category;
```

```
INFO  : Executing command(queryId=hive_20260929132013_d06e7163-d49c-4a01-897f-87019649d8da): SELECT
    product_category,
    SUM(total_amount) AS total_revenue,
    ROUND(
        SUM(total_amount) / COUNT(DISTINCT order_id),
        2
    ) AS average_order_value
FROM cleaned_transactions
WHERE transaction_id IS NOT NULL
GROUP BY product_category
ORDER BY product_category
WARN  : Hive-on-MR is deprecated in Hive 2 and may not be available in the future versions. Consider using a different execution engine (i.e. tez) or using Hive 1.X rele
ses.
INFO  : Query ID = hive_20260929132013_d06e7163-d49c-4a01-897f-87019649d8da
INFO  : Total jobs = 2
INFO  : Launching Job 1 out of 2
INFO  : Starting task [Stage-1:MAPRED] in serial mode
INFO  : Number of reduce tasks not specified. Estimated from input data size: 1
INFO  : In order to change the average load for a reducer (in bytes):
INFO  :   set hive.exec.reducers.bytes.per.reducer=<number>
INFO  : In order to limit the maximum number of reducers:
INFO  :   set hive.exec.reducers.max=<number>
INFO  : In order to set a constant number of reducers:
INFO  :   set mapreduce.job.reduces=<number>
INFO  : number of splits:1
INFO  : Submitting tokens for job: job_local711865561_0007
INFO  : Executing with tokens: []
INFO  : The url to track the job: http://localhost:8080/
INFO  : Job running in-process (local Hadoop)
INFO  : 2026-09-29 13:20:15,306 Stage-1 map = 100%,  reduce = 100%, Cumulative CPU 0.29 sec
INFO  : MapReduce Total cumulative CPU time: 290 msec
INFO  : Ended Job = job_local711865561_0007
INFO  : Launching Job 2 out of 2
INFO  : Starting task [Stage-2:MAPRED] in serial mode
INFO  : Number of reduce tasks determined at compile time: 1
INFO  : In order to change the average load for a reducer (in bytes):
INFO  :   set hive.exec.reducers.bytes.per.reducer=<number>
INFO  : In order to limit the maximum number of reducers:
INFO  :   set hive.exec.reducers.max=<number>
```

Figure 9. Category-wise revenue query executed as two MapReduce jobs.

```
INFO  : Total MapReduce CPU Time Spent: 620 msec
INFO  : Completed executing command(queryId=hive_20260929132013_d06e7163-d49c-4a01-897f-87019649d8da); Time taken: 3.241 seconds
+--------------------+----------------+----------------------+
| product_category   | total_revenue  | average_order_value  |
+--------------------+----------------+----------------------+
|                    | 600.0          | 600.0                |
| Clothing           | 1700.0         | 566.67               |
| Electronics        | 3700.0         | 925.0                |
| Groceries          | 950.0          | 475.0                |
| Unknown            | NULL           | NULL                 |
| product_category   | NULL           | NULL                 |
+--------------------+----------------+----------------------+
6 rows selected (3.682 seconds)
```

Figure 10. Query output showing revenue and average order value per category.

Final Results

The final output of the category-wise query is summarized below.

Product Category	Total Revenue	Average Order Value
Electronics	3700.0	925.00
Clothing	1700.0	566.67
Groceries	950.0	475.00
(Blank category)	600.0	600.00
Total Sales Revenue	**6950.0**	

Summary

This assignment showed how Apache Hive gives structured SQL access to raw CSV data stored in HDFS. The work was completed in the following stages:

- A Hadoop and Hive cluster (namenode, datanode and HiveServer2) was started with Docker Compose, and HDFS was confirmed healthy.
- The transactions CSV file was uploaded to HDFS and mapped to an **external Hive table**, so the raw data stayed untouched in HDFS.
- Missing values (a blank quantity, category and unit price) were handled through a **cleaned view** built on top of the raw table.
- HiveQL queries calculated a **total sales revenue of 6950.0**.
- Category-wise results: **Electronics** 3700.0 (average order value 925.00), **Clothing** 1700.0 (566.67) and **Groceries** 950.0 (475.00), with 600.0 from a transaction whose category was blank.

Electronics is the highest-earning category. Overall, the exercise shows that Hive lets a business intelligence team analyze revenue patterns with familiar SQL-style queries, without moving data out of HDFS or writing MapReduce code by hand.